

Method 1: Using SQLite Database related CRUD functionalities

To performing CRUD (Create, Read, Update, Delete) operations using an HTML template in **Flask** typically involves:

1. Setting up Flask routes
2. Connecting to a database (e.g., SQLite)
3. Rendering templates (HTML with Jinja2)
4. Using forms for user interaction

Folder Structure (Example)

```
/crud_app
├── app.py
├── templates/
│   ├── index.html
│   ├── create.html
│   └── update.html
└── database.db
```

1. Install Flask

```
pip install flask
```

2. Basic Flask App (app.py)

```
from flask import Flask, render_template, request, redirect, url_for
import sqlite3

app = Flask(__name__)

# Initialize DB
def init_db():
    with sqlite3.connect("database.db") as conn:
        conn.execute('''
            CREATE TABLE IF NOT EXISTS students (id INTEGER PRIMARY KEY
AUTOINCREMENT,
            name TEXT NOT NULL,
            email TEXT NOT NULL UNIQUE)''')

init_db()

@app.route('/')
def index():
    with sqlite3.connect("database.db") as conn:
        cur = conn.cursor()
```

```

        cur.execute("SELECT * FROM students")
        students = cur.fetchall()
        return render_template('index.html', students=students)

@app.route('/create', methods=['GET', 'POST'])
def create():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        with sqlite3.connect("database.db") as conn:
            conn.execute("INSERT INTO students (name, email) VALUES (?, ?)",
                (name, email))
            return redirect(url_for('index'))
    return render_template('create.html')

@app.route('/update/<int:id>', methods=['GET', 'POST'])
def update(id):
    with sqlite3.connect("database.db") as conn:
        cur = conn.cursor()
        if request.method == 'POST':
            name = request.form['name']
            email = request.form['email']
            cur.execute("UPDATE students SET name=?, email=? WHERE id=?", (name,
                email, id))
            conn.commit()
            return redirect(url_for('index'))
        cur.execute("SELECT * FROM students WHERE id=?", (id,))
        student = cur.fetchone()
        return render_template('update.html', student=student)

@app.route('/delete/<int:id>')
def delete(id):
    with sqlite3.connect("database.db") as conn:
        conn.execute("DELETE FROM students WHERE id=?", (id,))
        return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True)

```

3. HTML Templates

templates/index.html

```

<!doctype html>
<html>
<head><title>Student List</title></head>

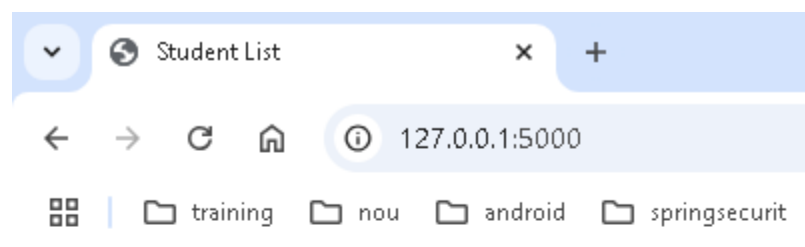
```

```

<body>
  <h2>Student List</h2>
  <a href="{ url_for('create') }">Add Student</a>
  <table border="1">
    <tr><th>ID</th><th>Name</th><th>Email</th><th>Actions</th></tr>
    {% for student in students %}
      <tr>
        <td>{{ student[0] }}</td>
        <td>{{ student[1] }}</td>
        <td>{{ student[2] }}</td>
        <td>
          <a href="{ url_for('update', id=student[0]) }">Edit</a> |
          <a href="{ url_for('delete', id=student[0]) }">Delete</a>
        </td>
      </tr>
    {% endfor %}
  </table>
</body>
</html>

```

Output:



Student List

[Add Student](#)

ID	Name	Email	Actions
2	Anjali Dhami	anjali@gmail.com	Edit Delete
3	Jasmir Chaudhary	jasmir@gmail.com	Edit Delete
4	Nabin Bhatta	nabin@gmail.com	Edit Delete

templates/create.html

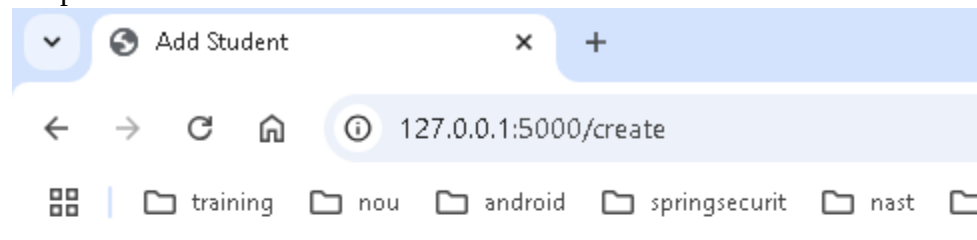
```
<!doctype html>
```

```

<html>
<head><title>Add Student</title></head>
<body>
  <h2>Add Student</h2>
  <form method="POST">
    Name: <input type="text" name="name" required><br>
    Email: <input type="email" name="email" required><br>
    <input type="submit" value="Add">
  </form>
  <a href="{{ url_for('index') }}">Back</a>
</body>
</html>

```

Output:



Add Student

Name:

Email:

[Back](#)

templates/update.html

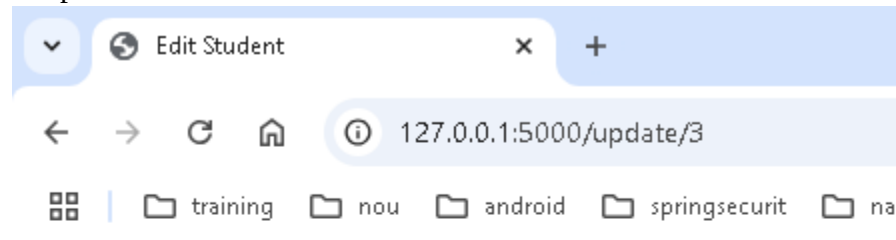
```

<!doctype html>
<html>
<head><title>Edit Student</title></head>
<body>
  <h2>Edit Student</h2>
  <form method="POST">
    Name: <input type="text" name="name" value="{{ student[1] }}"
required><br>
    Email: <input type="email" name="email" value="{{ student[2] }}"
required><br>
    <input type="submit" value="Update">
  </form>

```

```
<a href="{{ url_for('index') }}">Back</a>
</body>
</html>
```

Output:



Edit Student

Name:

Email:

[Back](#)

Method 2: Using MYSQL Database to perform CRUD operations

To perform CRUD operations in **Flask** using **MySQL** and HTML templates, follow this guide.

1. Install required libraries:

pip install flask flask-mysqldb

2. MySQL Database Setup:

Create a database and table using a MySQL client (e.g., MySQL Workbench or terminal):

```
CREATE DATABASE flask_crud;
```

```
USE flask_crud;
```

```
CREATE TABLE students ( id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100)
NOT NULL, email VARCHAR(100) NOT NULL );
```

app.py — Flask with MySQL

```
from flask import Flask, render_template, request, redirect, url_for
from flask_mysqldb import MySQL
```

```

app = Flask(__name__)

# Configure MySQL
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = ''
app.config['MYSQL_DB'] = 'flask_crud'

mysql = MySQL(app)

@app.route('/')
def index():
    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM students")
    data = cur.fetchall()
    cur.close()
    return render_template('index.html', students=data)

@app.route('/create', methods=['GET', 'POST'])
def create():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        cur = mysql.connection.cursor()
        cur.execute("INSERT INTO students (name, email) VALUES (%s, %s)", (name,
email))
        mysql.connection.commit()
        cur.close()
        return redirect(url_for('index'))
    return render_template('create.html')

@app.route('/update/<int:id>', methods=['GET', 'POST'])
def update(id):
    cur = mysql.connection.cursor()
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        cur.execute("UPDATE students SET name=%s, email=%s WHERE id=%s", (name,
email, id))
        mysql.connection.commit()
        cur.close()
        return redirect(url_for('index'))

    cur.execute("SELECT * FROM students WHERE id=%s", (id,))
    student = cur.fetchone()

```

```

        cur.close()
        return render_template('update.html', student=student)

@app.route('/delete/<int:id>')
def delete(id):
    cur = mysql.connection.cursor()
    cur.execute("DELETE FROM students WHERE id=%s", (id,))
    mysql.connection.commit()
    cur.close()
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True)

```

Method 3: Using SQLAlchemy (an ORM tool)

Using **SQLAlchemy ORM** in Flask is cleaner, more secure, and scalable.

Here's how to perform **CRUD operations** in Flask using **MySQL with SQLAlchemy ORM** and HTML templates.

1. Install Required Packages:

pip install flask_sqlalchemy pymysql

2. Create MySQL Database:

CREATE DATABASE flask_crud;

3. Project Structure

/flask_crud_app

```

|
|— app.py
|— templates/
|   |— index.html
|   |— create.html
|   |— update.html

```

app.py — Flask + SQLAlchemy ORM

```

from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

```

```

# MySQL connection (using PyMySQL driver)
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:@localhost/flask_crud'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# Model
class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), nullable=False)

    def __repr__(self):
        return f"<Student {self.name}>"

# READ
@app.route('/')
def index():
    students = Student.query.all()
    return render_template('index.html', students=students)

# CREATE
@app.route('/create', methods=['GET', 'POST'])
def create():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        new_student = Student(name=name, email=email)
        db.session.add(new_student)
        db.session.commit()
        return redirect(url_for('index'))
    return render_template('create.html')

# UPDATE
@app.route('/update/<int:id>', methods=['GET', 'POST'])
def update(id):
    student = Student.query.get_or_404(id)
    if request.method == 'POST':
        student.name = request.form['name']
        student.email = request.form['email']
        db.session.commit()
        return redirect(url_for('index'))

```



```

        return render_template('update.html', student=student)

# DELETE
@app.route('/delete/<int:id>')
def delete(id):
    student = Student.query.get_or_404(id)
    db.session.delete(student)
    db.session.commit()
    return redirect(url_for('index'))

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(debug=True)

```

HTML Templates (Minimal)

templates/index.html

```

<!doctype html>
<html>
<head><title>Student List</title></head>
<body>
    <h2>Student List</h2>
    <a href="{{ url_for('create') }}">Add Student</a>
    <table border="1">
        <tr><th>ID</th><th>Name</th><th>Email</th><th>Actions</th></tr>
        {% for student in students %}
        <tr>
            <td>{{ student.id }}</td>
            <td>{{ student.name }}</td>
            <td>{{ student.email }}</td>
            <td>
                <a href="{{ url_for('update', id=student.id) }}">Edit</a> |
                <a href="{{ url_for('delete', id=student.id) }}">Delete</a>
            </td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>

```

templates/create.html

```
<!doctype html>
<html>
<head><title>Add Student</title></head>
<body>
  <h2>Add Student</h2>
  <form method="POST">
    Name: <input type="text" name="name" required><br>
    Email: <input type="email" name="email" required><br>
    <input type="submit" value="Add">
  </form>
  <a href="{{ url_for('index') }}">Back</a>
</body>
</html>
```

templates/update.html

```
<!doctype html>
<html>
<head><title>Edit Student</title></head>
<body>
  <h2>Edit Student</h2>
  <form method="POST">
    Name: <input type="text" name="name" value="{{ student.name }}"
required><br>
    Email: <input type="email" name="email" value="{{ student.email }}"
required><br>
    <input type="submit" value="Update">
  </form>
  <a href="{{ url_for('index') }}">Back</a>
</body>
</html>
```